AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Amrita School of AI
COIMBATORE CAMPUS

CASE STUDY 2 · BEYOND THE SYLLABUS

A digital twin on Azure

Where production rigour comes from, and what it costs.

Dr. Abhijith Anandakrishnan · Assistant Professor

24AIM332 · Odd Semester 2026-27

DISCLOSURE

Same boundary as before

A consulting engagement I led. Architecture and engineering lessons only.

The interesting content here is the **failures**, because they are what a textbook cannot give you.

> No client identity beyond what is already public, no data, no commercial terms.

WHAT A DIGITAL TWIN IS

A live model of a real thing

A structured representation of physical entities and their relationships, kept current, that you can query and simulate against.

The cloud part is unremarkable. The **discipline** part is where projects fail.

Components: - A **model** of entity types and relations - A **state** plane holding current values - A **history** you can replay - A **simulation** layer that branches and scores

The whole system is declared in Bicep.

Templates, parameter files per environment, and a bootstrap template that creates what the main one needs.

CUSTOM DOMAINS

The three-stage thing everyone gets wrong once

Binding a custom domain with a managed certificate is **not** a switch.

The certificate is validated against a hostname that must **already exist**.

So the order is: create the binding unsecured → issue the certificate against it → bind the certificate.

> Try to do it in one step and it fails in a way that reads like a permissions problem.

OBSERVABILITY

One call, or the service is half-instrumented

Tracing, structured logging and metrics are configured **together**, at startup, in a single call.

> If they are three separate optional setups, some service will have two of the three, and you will discover which one during an incident. Log lines carry their trace ID. That is what makes them useful.

HEALTH

Liveness and readiness are different questions

Liveness — is the process alive?

Readiness — can it serve traffic right now?

Plus a **preflight** command that answers: *would a demonstration work if it started now?* It names what is broken.

It runs in the pipeline, so it stays exercised rather than rotting.

THE PART WORTH THE WHOLE LECTURE

Apply the template twice. The second time must change nothing.

If it does change something, the deployment never converges, and you find out during an incident.

TWO FINDINGS, ON FIRST DEPLOYMENT

One real defect, one piece of noise

A real defect

A budget was declared with a bare date, which the service silently normalises to a different form.

Every apply saw a difference. The deployment would **never** have converged.

Noise

A workbook property that the what-if API cannot read at all.

Not a defect. Excluded **narrowly**, by name.

Keeping those two apart is why the assertion is still worth running.

Excluding broadly to make the check pass turns a test into decoration.

TEARDOWN

Ordering is not a detail

Deleting the resource group directly, while a bound certificate existed, **stranded** it:

the certificate reported in use, the app failed, the environment returned an internal error, and the group rejected the very updates that would have fixed it.

COST: OVER AN HOUR

Teardown now unbinds domains and deletes certificates **first**.

A clean removal takes about eight minutes.

THE PIPELINE CORRECTION, TWICE

What should merging do?

First attempt

The infra workflow applied on **every merge**, silently rebuilding whatever had just been torn down.

Overcorrection

Made merging **plan-only**.

Cheap, and it traded away the only thing worth having: building it is the only way to know the template works.

Where it landed

A merge **builds for real**, asserts convergence, and removes it in the same run.

A **failed** run leaves it standing, because that is the one case worth paying to inspect.

Quality and rigour come before cost.

But only just: the successful path still tears everything down.

A FINDING WORTH CARRYING

The graph that lied politely

The managed graph service holds **current state only**.

Pass it an as-of timestamp it cannot honour, and it returns present state dressed as history.

Silently. And biased towards looking better informed than the system actually was.

> An evaluation built on that would have scored itself far too well, and nobody would have known why. It now **refuses** such reads.

A component that cannot answer a question must refuse it.

Returning a plausible wrong answer is worse than an error, because errors get fixed.

LOCAL PARITY

Three commands, same as CI

```
make cloud-verify
make cloud-up
make cloud-down
```

The same three shapes CI runs.

If the local path and the CI path differ, the pipeline becomes something you debug rather than something you trust.

WHAT TO TAKE AWAY

Five transferable points

1. Assert idempotence. Exclude noise narrowly, never broadly.

2. Teardown has an order. Discover it deliberately, not during a bill.

3. Configure observability in one call, or it will be partial.

4. A component that cannot answer must refuse, not approximate.

5. Build for real in CI. A plan-only pipeline proves the plan, not the system.

Every one of these was learned by getting it wrong first.

That is what a case study is for.